פיתוח עם סוכני-קוד

פיתוח ב־Vibe Coding עם Claude Code
ועם ChatGPT / Codex

אפיון, תיעוד ובדיקות שמאפשרים להמשיך גם מחר



מגבלה ידועה - לסוכן יש תמיכה בגודל קונטקסט מוגדר:

לדוגמא - לסוכן Claude Opus 5.5

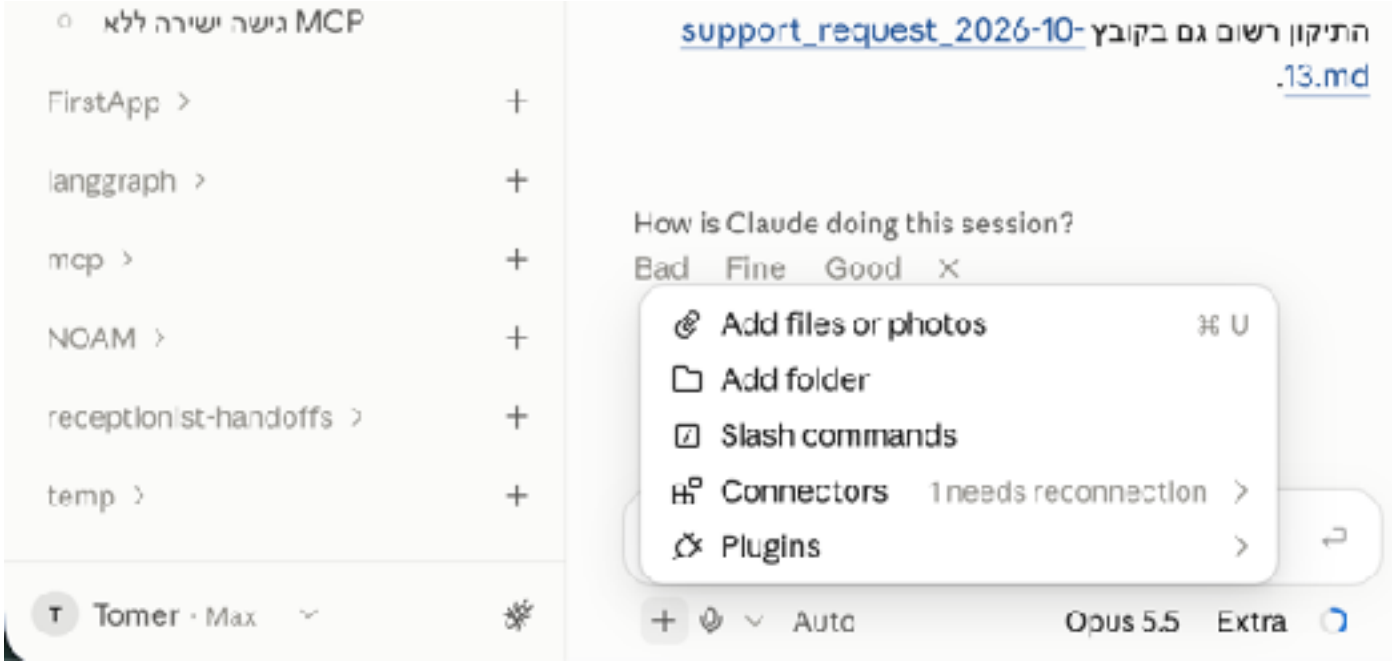
יש קונטקסט בגודל 1M טוקנים

אחרי ~1M טוקנים - הסוכן מבצע Compactization:

סיכום ומחיקה חלקית של הקונטקסט.

השיטה להתגבר על כך:
קבצים בתיקיה שישמשו כContext

סוכן ותיקייה: מתחילים בקלוד



בתפריט + בוחרים Add folder ומחברים את תיקיית הפרויקט. אפשר גם בלי VS Code.

סוכן ותיקייה: מתחילים בקלוד

סוכן ותיקייה: על אותה תיקיה אפשר לעבוד עם Codex

מה ההבדל בין מודל שפה לסוכן?

מודל שפה: LLM

מבין הקשר ומייצר תשובה או קוד. לבדו הוא אינו קורא או משנה את התיקייה.

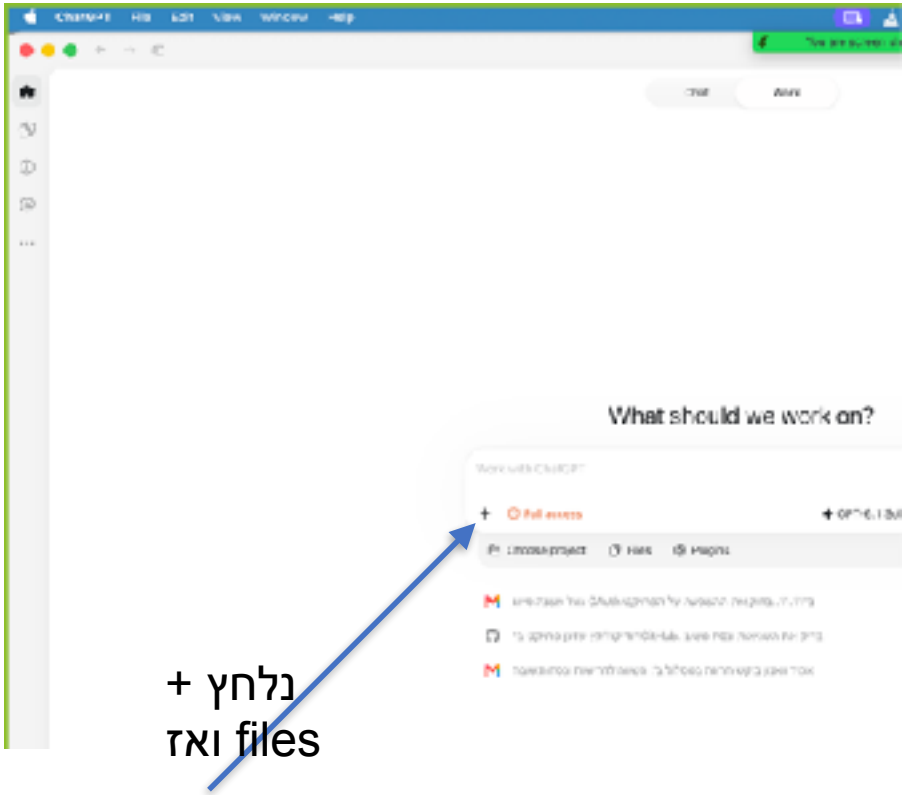
סוכן: מודל בתוך סביבת עבודה

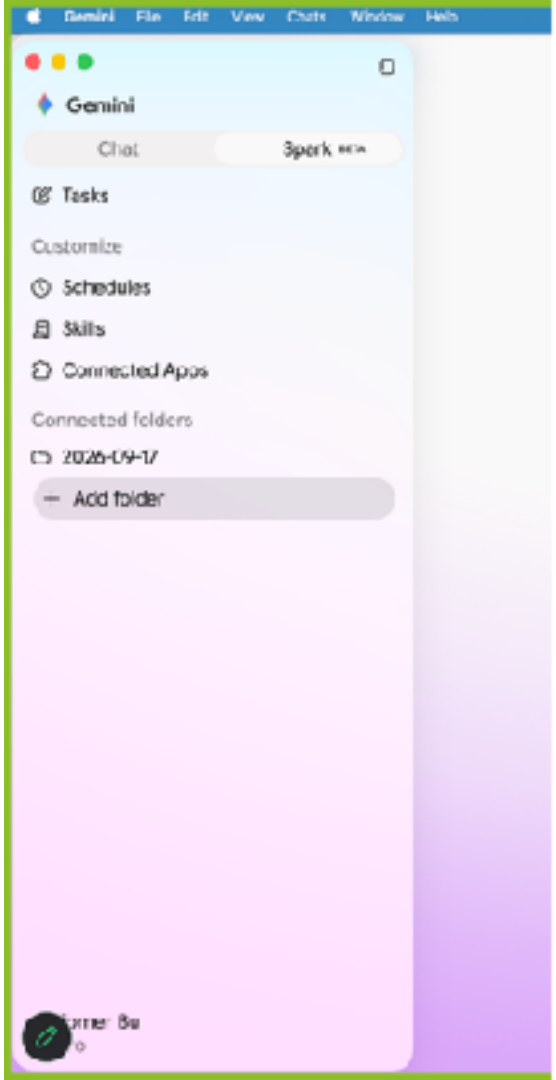
בוחר כלי, מפעיל אותו דרך הסביבה, קורא את התוצאה ומחליט איך להמשיך.

הכלים שעומדים לרשותו

קבצים ותיקיות, טרמינל ובדיקות, וגם חיבורי MCP לקבצים ולשירותים.

מה ההבדל בין מודל שפה לסוכן?





מה ההבדל בין מודל שפה לסוכן?

מה נוכל לבצע בסוף השיעור



להתחיל פרויקט

לפתוח תיקייה, לחבר סוכן ולהפעיל את הפרויקט מקומית.

לפתח פיצ׳ר שאפשר לבדוק

לנסח תנאי קבלה למשימה ולעבוד במחזור של בדיקה ומימוש.

להמשיך בשיחה חדשה

להעביר לסוכן הבא אפיון, החלטות ומצב עבודה מעודכן.

תשתיות לפיתוח עצמאי

שכבת היישום: Next.js

ממשק המשתמש ולוגיקת השרת של היישום.

שכבת הנתונים: Supabase

מסד PostgreSQL, התחברות משתמשים, הרשאות ואחסון קבצים.

פריסה והרצה: Vercel

אירוח היישום, גרסאות תצוגה מקדימה ופריסה מתוך מאגר הקוד.

כשאתם בוחרים את הכלים ומחברים אותם באמצעות סוכנים, השמיים הם הגבול.

כלים מקומיים וחיבורי MCP

עבודה בתיקייה

הסוכן מחפש, קורא, יוצר ועורך קבצים. הוא יכול להריץ פקודות ובדיקות.

חיבורי MCP

מחברים כלים נוספים לעבודה על מסמכים וקבצים, מסדי נתונים ושירותים.

בדיקת היכולת בפועל

מוודאים איזה כלי מחובר, לאילו קבצים הוא ניגש ואיזו פעולה הוא מאפשר.

מה מוסיף לנו העורך VS Code?

מעבר נוח בין סוכנים

פותחים את אותה תיקייה עם תוספים כמו Claude Code ו־Codex.

כלים לעבודה עם קבצים

תוספים לתצוגת Markdown, לעבודה עם נתונים, לעיצוב קוד ולהשוואת שינויים.

סביבת עבודה מרוכזת

קבצים, חלונית הסוכן, טרמינל ובקרת גרסאות באותו חלון.

בוחרים את הסביבה שנוחה לנו. הידע והתוצרים נשארים בתיקייה.

התקנת העורך VS Code

הורדה

נכנסים לאתר code.visualstudio.com/download ובוחרים את מערכת ההפעלה.

התקנה

ב־Windows מפעילים את המתקין. ב־Mac מעבירים את היישום לתיקיית Applications.

פתיחה ראשונה

מפעילים את העורך ופותחים את תיקיית הקורס דרך File ואז Open Folder.

במהלך הקורס נשתמש בעורך לעבודה עם סוכנים, קבצים ותוספים.

התקנת סביבת Antigravity

הורדה

נכנסים לאתר antigravity.google/download ובוחרים Antigravity IDE.

התקנה והתחברות

מורידים את הגרסה המתאימה למחשב, מתקינים ומתחברים לפי ההנחיות.

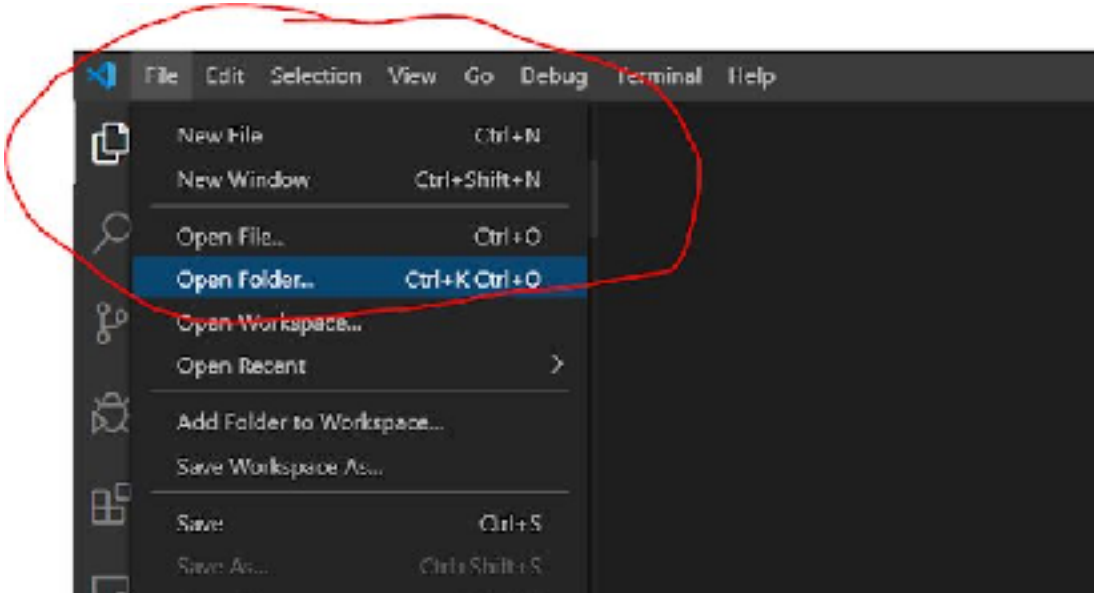
תיקיית העבודה

פותחים את אותה תיקיית פרויקט ומתחילים לעבוד עם הסוכן מתוך העורך.

נתנסה בשתי הסביבות במהלך הקורס, עם אותם קבצים ותיעוד.

פתיחת התיקייה ב־VS Code

בתפריט File בוחרים Open Folder...



בוחרים את התיקייה

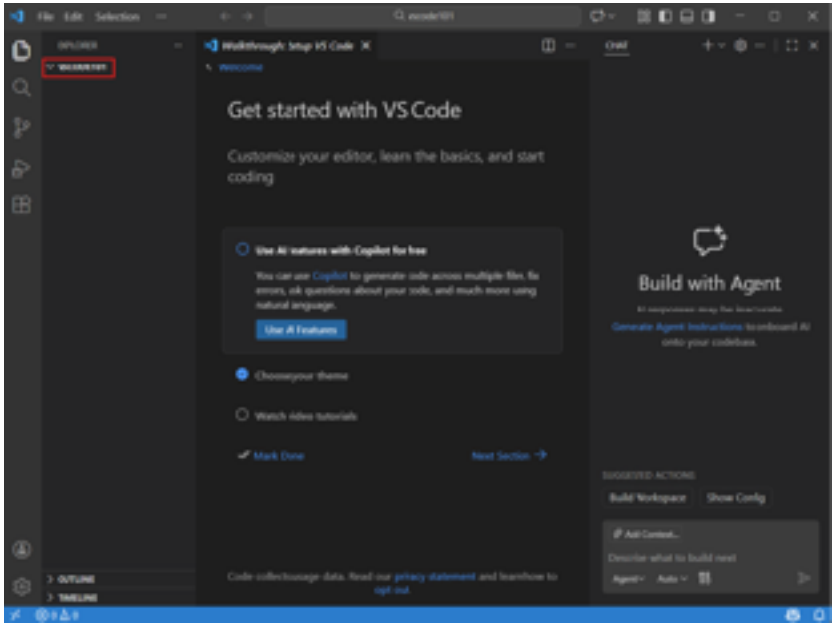
פותחים את my-project כולה, כדי שהסוכן יראה את הפרויקט.

בודקים אמון בתיקייה

מאשרים אמון רק בתיקייה שיצרנו או שמקורה מוכר לנו.

צילום הדרכה ב־Windows מגרסה קודמת. ב־Mac התפריט נמצא בראש המסך.

היכרות עם חלון הקבצים



חלון הקבצים: Explorer

כאן רואים את תיקיית הפרויקט ואת הקבצים שבתוכה.

הקובץ README.md

נכתוב מה הפרויקט עושה ואיך מפעילים אותו.

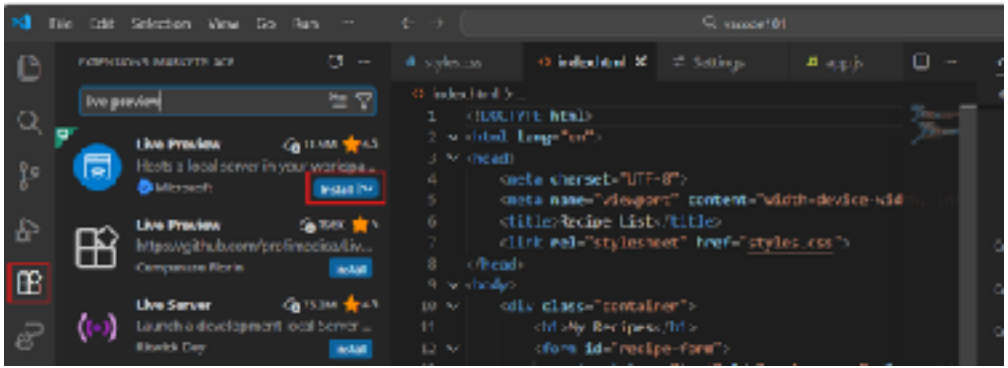
תיקייה שלמה

קבצים נלווים מאפשרים לסוכן להבין את מבנה הפרויקט.

התקנת תוסף הסוכן

פותחים את Extensions

ב־Windows: Ctrl+Shift+X
ב־Mac: Cmd+Shift+X

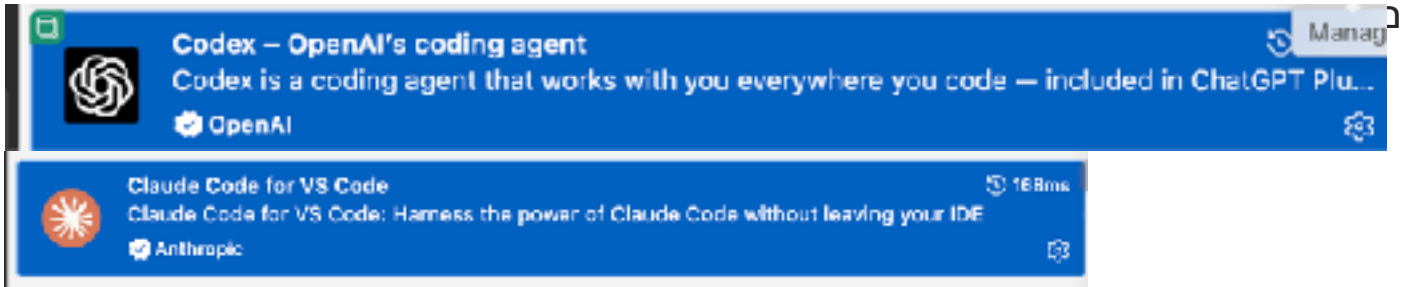


בוחרים תוסף רשמי

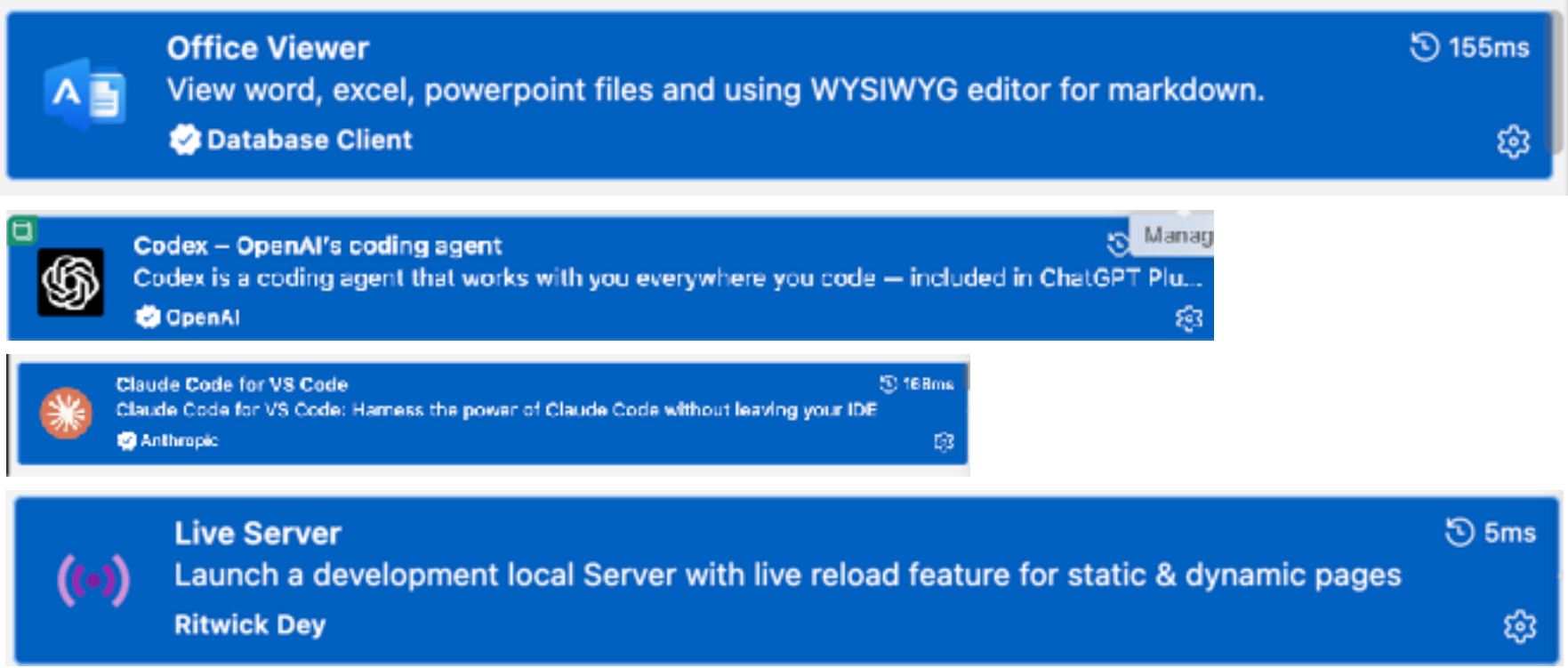
הכלי Claude Code מאת Anthropic
הכלי Codex מאת OpenAI

מתחברים לחשבון

לוחצים Install, פותחים ומשלימים Sign in.



תוספים מומלצים:



צפייה חיה ב־HTML עם Live Server

התקנת התוסף

בחלון Extensions מחפשים Live Server מאת Ritwick Dey ולוחצים Install.

צפייה בדף

פותחים index.html בתיקיית הפרויקט ובוחרים Open with Live Server.

בדיקה קצרה

משנים כותרת ושומרים. רואים את העדכון בדפדפן דרך השרת המקומי.

לתצוגת HTML משתמשים ב־Live Server. לפרויקט Next.js מפעילים npm run dev.

צפייה במסמכים עם Office Viewer

התקנת התוסף

בחלון Extensions מחפשים Office Viewer מאת cweijan ולוחצים Install.

פתיחת תוצרי הסוכן

פותחים קובץ DOCX, XLSX, PPTX או PDF מתוך תיקיית הפרויקט.

בחירת התצוגה

אם צריך, בוחרים Reopen Editor With ואת הצופה המתאים לסוג הקובץ.

בדיקת ההתקנה: פותחים מסמך מהתיקייה ורואים את התוכן בתוך העורך.

פרומפט להקמת פרויקט: חלק א׳

פעל כסוכן פיתוח שמלווה אותי מהרעיון לגרסה ראשונה עובדת.
המוצר שאני רוצה לבנות: [תיאור הרעיון וקהל היעד].
הפעולה המרכזית שהמשתמש צריך לבצע: [תיאור הפעולה].
התחל בהבנת הדרישה ובסקירת החומרים שסיפקתי בתיקייה.
קרא הנחיות קיימות, קוד ומסמכים. שמור עבודה קיימת.
הצג מה הבנת, מה כבר קיים ואילו פרטים דרושים להחלטה.
שאל שאלות ממוקדות רק כשחסר מידע שמשנה את המוצר.
הצע גרסה ראשונה ממוקדת, כולל מה יישאר לשלב הבא.
הצע שימוש ב־Next.js, ב־Supabase וב־Vercel לפי הצורך.
הסבר מה כל כלי ייתן למוצר ואילו חיבורים נוספים יקדמו אותו.
בדוק אילו כלים וחיבורי MCP זמינים לך לעבודה על החומרים.

צור לי משחק סוליטייר עם צד שרת בnext על גבי vercel (cloud)
לגבי דטהבייס - נשתמש בסופהבייס

database

מחשב שמותקן עליו
דטהבייס

צד שרת - ייכתב עם הספריה nextjs

את צד שרת נאחסן בענן

זה הענן Vercel
שיחזיק את צד שרת וצד לקוח

פרומפט להקמת פרויקט: חלק ב׳

ערוך את הקובץ md - כך ש:

זהו פרוייקט לעבודה עם מספר סוכנים
מקור האמת הוא הקבצים בלבד!

שיחה לא תשאר בצ׳אט - יש לתעד בקשות והחלטות וכל משימה שאנו מבצעים

אנחנו רוצים לעבוד כך:

לפני כל משימה - לתעד את המשימה היטב בתיקיית משימות עם תאריך וזמן
אחרי כל משימה - לתעד את המסכים ואת התוצרים ואת הקבצים כדי

התיקיות בפרוייקט:

docs
progress
issues
src

קבצים בפרוייקט:
readme
agents
claude

כל סוכן חדש שמצטרף צריך לקרוא את התיעוד

קבצים בפרוייקט:

CLAUDE.md



agents.md

readme.md

זה הקובץ שקלוד קורא בתחילת עבודה על פרוייקט

זה קובץ שמיועד לכל הסוכנים בפרוייקט

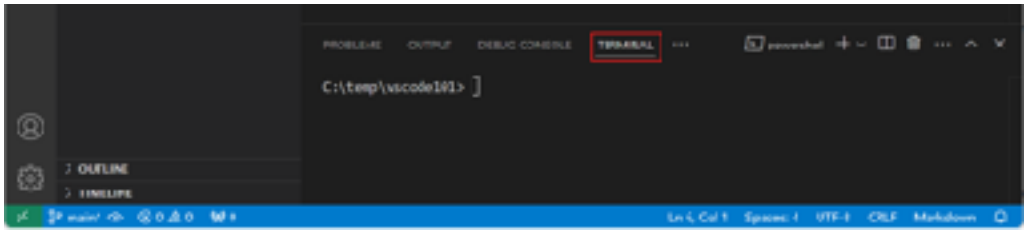
זה קובץ שמיועד לכל בני האדם בפרוייקט

פרומפט להקמת פרויקט: חלק ב׳

כתוב בקובץ docs/spec.md את המטרה, קהל היעד וגבולות הגרסה.
הוסף תרחישי שימוש, נתונים, הרשאות ותנאי קבלה מדידים.
שמור החלטות והנמקות בקובץ docs/decisions.md.
צור הוראות עבודה ב־AGENTS.md והתאם אותן לכלי שבו אנו עובדים.
בשימוש ב־Claude Code הוסף CLAUDE.md שמפנה להנחיות המשותפות.
תעד בקובץ README.md התקנה, הפעלה ופקודות בדיקה.
חלק את העבודה למשימות קטנות ושמור אותן ב־docs/status.md.
הצג לי את האפיון ואת התוכנית לקבלת החלטה לפני המימוש.
לאחר אישור, בנה תרחיש שימוש שלם אחד עם נתוני דוגמה.
כתוב בדיקות להתנהגות, הרץ אותן ובדוק את התוצאה בממשק.
בסיום דווח מה עובד, מה נבדק בפועל ומה המשימה הבאה.

הפעלת הפרויקט במחשב

בתפריט Terminal בוחרים New Terminal




```
npm install
npm run dev
```

אחרי שהסוכן יצר את הפרויקט:
מתקינים תלויות, מפעילים ופותחים
את הכתובת שמופיעה בטרמינל.
בדרך כלל: http://localhost:3000

פרק ב׳

הזיכרון של הפרויקט

איך הסוכן של מחר ממשיך את העבודה של היום?



כשפותחים שיחה חדשה

ההחלטה של אתמול

בחרנו התנהגות רצויה, הגדרנו גבולות וסיימנו חלק מהמשימה.

הבקשה של היום

״תמשיך מאיפה שעצרנו״. הסוכן החדש צריך לדעת איפה עצרנו ולמה.

המידע שצריך להישאר

אפיון מעודכן, החלטות, מה נבדק ומה עדיין פתוח.

התיקייה צריכה לאפשר המשך עבודה גם ללא היסטוריית השיחה.

קובצי Markdown והקונטקסט



ידע מחוץ לשיחה

קובצי טקסט בסיומת md נשארים בתיקייה אחרי סיום השיחה.

טעינה לפי המשימה

הסוכן קורא את ההנחיות ואת חלקי האפיון הרלוונטיים.

תחזוקה במקום הצטברות

מעדכנים החלטות ומצב. מסירים מידע שהתיישן.

קבצים מאפשרים לשחזר הקשר. מגבלת הקונטקסט עדיין קיימת.

שלושה קבצים עם תפקידים שונים

הקובץ README.md

מה הפרויקט עושה, איך מתקינים, איך מפעילים ואיך מריצים בדיקות.

הקובץ AGENTS.md

כללי עבודה לסוכן: מקורות מידע, גבולות השינוי ופקודות אימות.

הקובץ CLAUDE.md

הנחיות ל־Claude Code. אפשר לייבא ממנו את ההנחיות המשותפות.

השם AGENTS.md מתייחס להוראות לסוכנים, לא להגדרת תת־סוכן.

מבנה תיקיית הפרויקט

```
my-project/
  README.md
  AGENTS.md
  CLAUDE.md
  docs/
    spec.md
    decisions.md
    status.md
  src/
  tests/
```

docs/spec.md :האפיון

מטרות הפרויקט, דרישות ותנאי קבלה.

docs/decisions.md :ההחלטות

מה בחרנו, למה, ומה ההשלכות.

docs/status.md :המצב

מה אומת, מה פתוח ומה הצעד הבא.

הנחיות משותפות לשני הכלים

בקובץ AGENTS.md

קרא תחילה את האפיון ואת מצב הפרויקט.
בצע שינוי התנהגותי אחד בכל פעם.
הגדר תנאי קבלה וכתוב בדיקה לפני המימוש.
הפעל את פקודות הבדיקה שב־README.
בסיום עדכן את האפיון ואת מצב הפרויקט.
דווח רק על בדיקות שהורצו בפועל.

בקובץ CLAUDE.md

```
@AGENTS.md
```

שומרים את הכללים במקום אחד.
ב־Claude Code אפשר להשתמש
בייבוא המפורש הזה.

כבר בשיחה הראשונה מבקשים מהסוכן לציין אילו קובצי הנחיות קרא.

פיתוח לפי אפיון: Spec-driven development

לפני המימוש

מנסחים דרישה, גבולות ותנאי קבלה שאפשר לבדוק.

בזמן הפיתוח

הסוכן מקשר בין הדרישה, הבדיקה והשינוי בקוד.

אחרי הפיתוח

בודקים תוצאות ומעדכנים את התיעוד ואת מצב המשימה.

האפיון הוא מקור האמת לדרישות. הקוד והבדיקות מראים מה קיים בפועל.

דרישה שאפשר לבדוק

הבקשה הכללית

"הוסף הנחה" אינה מגדירה למי מגיעה הנחה או כיצד מחשבים אותה.

תנאי קבלה

לדוגמה בחנות: הנחה של 10% מסכום של 200 ₪, עד תקרה של 50 ₪.

לפני המימוש

מבהירים חריגים ושומרים את הכללים באפיון. אחר כך כותבים בדיקות.

פיתוח עם בדיקות: TDD

בדיקה נכשלת: Red

כותבים בדיקה להתנהגות הרצויה ומריצים. מוודאים שהיא נכשלת מהסיבה הצפויה.

מימוש עובד: Green

ממשים את הכלל הדרוש ומריצים שוב עד שהבדיקה עוברת.

שיפור הקוד: Refactor

מסדרים את הקוד בלי לשנות את ההתנהגות ושוב מריצים את הבדיקות.

איך יודעים שהסוכן הצליח?

התנהגות צפויה

בודקים מקרה רגיל מול התוצאה שהאפיון דורש.

גבולות וכישלונות

בודקים ערך גבולי, קלט שגוי ופעולה שאינה מורשית.

ראיות לביצוע

מבקשים את הפקודה שהורצה, תוצאתה והחלקים שעדיין לא אומתו.

הודעת ״סיימתי״ של הסוכן צריכה להישען על בדיקה בפועל.

פרומפט לפיתוח בשיטת TDD: חלק א׳

פעל כסוכן פיתוח בפרויקט הנוכחי והשלם את המשימה הבאה:
השינוי המבוקש: [תיאור ההתנהגות הרצויה].
תנאי הקבלה: [תוצאה צפויה, גבולות ומקרי כישלון].
קרא את הנחיות הסוכן, את README.md ואת docs/spec.md.
עיין גם ב־docs/decisions.md וב־docs/status.md ובקוד הרלוונטי.
התיעוד הוא מקור האמת לדרישות, ולא היסטוריית הצ׳אט.
אם הדרישה החדשה סותרת את האפיון, הצג את הסתירה וברר אותה.
תעד החלטה שאושרה ועדכן את האפיון לפני המימוש.
סכם את ההתנהגות שיש להוסיף ואת הדרך שבה תבדוק אותה.
חלק את המשימה לשינויים קטנים שאפשר לבדוק בנפרד.
שמור על מוסכמות הפרויקט ועל שינויים קיימים שאינם חלק מהמשימה.

פרומפט לפיתוח בשיטת TDD: חלק ב׳

לכל שינוי התנהגותי, כתוב תחילה בדיקה שנגזרת מתנאי הקבלה.
הרץ אותה וודא שהיא נכשלת בגלל ההתנהגות החסרה.
ממש את השינוי הדרוש, הרץ שוב וודא שהבדיקה עוברת.
שפר את הקוד והריץ בדיקות נוספות שעלולות להיות מושפעות.
אל תמחק ואל תחליש בדיקה כדי להציג הצלחה.
הפעל תת־סוכן לסקירה עצמאית אם אפשר להגדיר לו משימה מועילה.
העבר לו את האפיון, הקבצים הרלוונטיים וגבולות הסקירה.
בדוק את הממצאים ואמת את התוצאה לאחר שילוב התיקונים.
עדכן את מסמכי הפרויקט כך שישקפו את המצב בסיום.
החזר סיכום: שינויים, פקודות ותוצאות בדיקה, פערים והצעד הבא.
אם בדיקה לא הורצה, ציין זאת ואת הסיבה במפורש.

פרק ג׳

עבודה עם תת־סוכנים

הגדרת משימות, כלים ובדיקת התוצרים



מהו תת־סוכן?



הסוכן הראשי

מבין את היעד, מחלק עבודה ובודק את התוצרים לפני שילובם.

משימה מצומצמת

למשל: סקירת השינוי מול האפיון, בלי לשנות קוד.

הגדרה מפורשת

תפקיד, מקורות, כלים מותרים ופורמט התשובה.

הגדרת תת־סוכן ב־Claude Code

בקובץ .claude/agents/spec-reviewer.md

```
---
name: spec-reviewer
description: Review changes against the spec
tools: Read, Grep, Glob
---
Read docs/spec.md and relevant code.
Check requirements and missing tests.
Do not edit files.
Return findings with file references.
```

השם מזהה את הסוכן.
התיאור מסביר מתי להפעיל אותו.
הכלים מגבילים את היכולות.
ההוראות מגדירות את התוצר.

בקשה: הפעל את הסוכן spec-reviewer לסקירת השינוי מול האפיון.

הגדרת תת־סוכן ב־Codex

בקובץ .codex/agents/spec-reviewer.toml

```
name = "spec_reviewer"
description = "Review changes against the spec"
sandbox_mode = "read-only"
developer_instructions = """
Read docs/spec.md and relevant code.
Check requirements and missing tests.
Do not edit files.
Return findings with file references.
"""
```

אותו תפקיד עסקי,
בפורמט ההגדרות של הכלי.

בקשה: השתמש בסוכן
spec_reviewer כדי לבדוק
את השינוי מול האפיון.

חלוקת עבודה בין סוכנים

המימוש

הסוכן הראשי מטפל בשינוי ובבדיקות ההתנהגות.

הסקירה

תת־סוכן קורא את האפיון ומחפש פערים, מקרי קצה והרשאות חסרות.

השילוב

הסוכן הראשי בודק את הממצאים ומאמת את התוצאה לאחר השילוב.

לכל תת־סוכן מגדירים משימה, גבולות ותוצר שאפשר לבדוק.

פרומפט להפעלת תת־סוכן לסקירה

הפעל תת־סוכן לסקירת השינוי הנוכחי מול האפיון.
מסור לו את ההנחיות הבאות ואת נתיבי הקבצים שהשתנו:
תפקידך לבדוק התאמה לדרישות, בלי לערוך קבצים.
קרא את docs/spec.md ואת החלטות הפרויקט הרלוונטיות.
בדוק את הקוד ואת הבדיקות עבור [המשימה ותנאי הקבלה].
חפש התנהגות חסרה, מקרי קצה ופערים בבדיקות או בהרשאות.
לכל ממצא, ציין קובץ, מיקום, דרישה והסבר להשפעה.
הבחן בין תקלה מוכחת, חשד ושאלה שמצריכה החלטה.
החזר ממצאים לפי חשיבות וציין גם מה לא יכולת לבדוק.
לאחר קבלת הסקירה, בדוק את הממצאים כסוכן הראשי.
תקן פערים מאומתים, הרץ בדיקות ודווח על התוצאה המשולבת.

הרשאות ובקרה בעבודה עם סוכן

גישה לפי המשימה

מגדירים אילו תיקיות, שירותים ופעולות דרושים לסוכן.

בדיקת השינויים

קוראים את ההבדלים ובודקים שהסוכן טיפל במשימה שהוגדרה.

נתונים וסודות

משתמשים בנתוני דוגמה ושומרים מפתחות מחוץ לקוד ולתיעוד משותף.

נקודת חזרה לפני שינוי



לפני המשימה

בודקים את מצב הקבצים ושומרים גרסה עובדת ב־Git.

אחרי השינוי

קוראים את ההבדלים: אילו קבצים השתנו והאם השינוי שייך למשימה?

לאחר האימות

שומרים Commit שמתאר את הפיצ׳ר ואת התיעוד המעודכן.

הפעלת היישום ברשת

חיבור הפרויקט ל־Vercel

מייבאים את מאגר הקוד ומוודאים שפקודת הבנייה מצליחה.

חיבור ל־Supabase

מגדירים משתני סביבה מתאימים, כתובות התחברות וסכמת נתונים.

בדיקה בכתובת Preview

בודקים את תרחישי השימוש מהאפיון עם נתוני דוגמה לפני פרסום.

הסוכן מבצע את הפריסה. אנחנו בודקים את התוצאה בכתובת שהתקבלה.

סיום יום העבודה

הושלם ואומת

מה השתנה, לאיזו דרישה הוא קשור ואיזו בדיקה עברה.

עדיין פתוח

מה חסר, מה לא נבדק והאם קיימת החלטה שצריך לקבל.

המשימה הבאה

מה הפעולה הבאה ואילו קבצים צריך לקרוא כדי לבצע אותה.

הקובץ docs/status.md נותן לסוכן הבא נקודת התחלה ברורה.

פרומפט להמשך עבודה בשיחה חדשה

המשך את הפרויקט מתוך הקבצים, ללא הנחה שיש לך שיחה קודמת.
קרא את הנחיות הסוכן, README.md ואת מסמכי האפיון וההחלטות.
קרא את docs/status.md ובדוק את מצב הקוד והבדיקות הרלוונטיות.
סכם בקצרה את מטרת המוצר, מה הושלם ומה נשאר פתוח.
הבחן בין מידע מתועד לבין הנחות ודברים שטרם אומתו.
אם הקוד והתיעוד סותרים זה את זה, הצג את הפער לפני שינוי הדרישה.
בחר את המשימה הבאה מתוך התוכנית והסבר למה היא הבאה בתור.
אם חסרה החלטה שמשנה את המוצר, שאל שאלה ממוקדת.
אחרת, בצע את המשימה בשיטת TDD לפי תנאי הקבלה באפיון.
בסיום עדכן את המצב, ההחלטות והתיעוד הרלוונטי.
דווח מה השתנה, מה נבדק בפועל והיכן הסוכן הבא צריך להמשיך.

תרגול: הסוכן של מחר

שלב ראשון: הגדרת המשימה

בוחרים שינוי קטן ושומרים אפיון, הנחיות ומצב עבודה.

שלב שני: פיתוח ובקרה

מממשים התנהגות אחת בשיטת TDD ומבקשים סקירה ממוקדת.

שלב שלישי: שיחה חדשה

מבקשים מסוכן חדש להסביר את המצב ולהציע את הצעד הבא מתוך הקבצים.

מבחן ההצלחה: המשך עבודה ללא הסבר מחדש של הפרויקט.

מה נשאר בתיקייה בסיום?



ידע שאפשר להעביר

אפיון עדכני, החלטות מתועדות והוראות הפעלה.

תוצר שאפשר לבדוק

קוד, בדיקות ותוצאות אימות שמקושרות לדרישות.

המשך עבודה ברור

מצב הפרויקט והמשימה הבאה, גם למי שלא קרא את השיחה.